

Intro. Dev. - TD9

Les algorithmes de tri

Une importante partie de ce TD se focalise sur l'application d'algorithmes de tri ou discute de leurs complexités, sans avoir nécessairement à produire du code. Pour chaque algorithme utilisé, on détaille les différentes étapes. Attention à ne pas faire trop de détails mais juste assez pour que l'on comprenne chaque étape du tri.

Exercice 1 : Tri All-star

On considère les tableaux suivants :

85	12	45	71	25	36	10	11	12	5	4	3
----	----	----	----	----	----	----	----	----	---	---	---

5	2	14	1	3	6	0	1	2	5	4	3
---	---	----	---	---	---	---	---	---	---	---	---

Triez ce tableau avec les algorithmes suivants :

- Algorithme de tri par insertion
- Algorithme de tri à bulles
- Algorithme de tri par sélection
- Algorithme de tri rapide
- Algorithme du tri fusion
- Algorithme de tri arborescent
- Algorithme de tri par tas

Si un algorithme vous pose problème, n'hésitez pas à vous entraîner sur d'autres tableaux (aléatoires ou donnés par l'enseignant) avant de passer aux exercices suivants

Exercice 2 : Tri par insertion

On considère les tableaux suivants :

85	58	45	71	25	16	27	11	10	5	4	25
----	----	----	----	----	----	----	----	----	---	---	----

5	20	32	45	36	31	52	58	20	56	59	67
---	----	----	----	----	----	----	----	----	----	----	----

Répondez aux questions suivantes :

1. Quel est le tableau le plus rapide (en nombre d'instructions exécutées) à trier parmi ces deux tableaux avec un tri par insertion ?
2. Déduisez quel est le meilleur cas d'un tri par insertion, et exprimez sa complexité
3. Quel est la complexité de l'algorithme (pire cas) ? Que ce passe-t-il si l'on utilise maintenant une dichotomie pour trouver la position d'insertion d'une valeur non triée dans l'ensemble trié ? Quel est la nouvelle complexité ? (indice : instructions d'affectation et instructions de comparaison)

Exercice 3 : Tri à bulles

Répondez aux questions suivantes :

1. Exprimez un pire cas pour le tri à bulles et exprimez le meilleur cas
2. Proposez une modification simple du tri à bulles permettant un meilleur cas en $O(n)$ (indice : pourquoi continuer le tri sur un tableau qui est déjà trié ?). Quel est le pire cas de cette version dite optimisée ?

Exercice 4 : Tri fusion

Répondez aux questions suivantes :

1. Exprimez la complexité pire cas, meilleur cas, et cas moyen pour le tri fusion.
2. Discutez de l'utilisation de listes chaînées lors de l'implémentation de cet algorithme : est-ce une bonne idée, que gagne-t-on ou perd-t-on ?

Exercice 5 : Tri rapide

Répondez aux questions suivantes :

1. Exprimez la complexité pire cas et meilleur cas ? Donnez aussi une description de ces cas
2. Il existe une variante permettant d'obtenir toujours la complexité du meilleur cas, trouvez le mécanisme de cette variante (indice : on ne choisit plus au hasard le pivot). A noter que cette variante qui est en théorie plus rapide que le tri rapide "classique" dans le pire cas, est en pratique moins efficace.

Exercice 6 : Tri arborescent

Répondez aux questions suivantes :

1. Exprimez la complexité pire cas et meilleur cas du tri arborescent. Donner un exemple de pire cas
2. Considérons maintenant l'utilisation d'un arbre binaire équilibré (AVL , Arbre rouge et noir, . . .) qui garanti un équilibre de l'arbre pour un temps de $O(n \log(n))$ avec chaque ajout. Quel est la complexité pire cas du tri arborescent si une telle structure est utilisée ?

Exercice 7 : Mémoire

Donner les complexité spatiales des différents tris vus en cours

Exercice 8 : Quelques algorithmes complémentaire

1. Codez le sous-programme permettant de tester si un tableau donné en paramètre est trié
2. Codez le sous-programme permettant d'insérer une valeur dans un tableau trié

Exercice 9 : Un peu de code

Écrivez le code des algorithmes de tri vus en cours